

TASK 5 – Perform Asymptotic Analysis

The proposed algorithms are **Linear Search, Binary Search, and Hash Search**. Their asymptotic growth shows how their performance changes as the number of student records n becomes very large.

1. Linear Search

Linear Search checks the records one by one.

- **Best case:** $\Omega(1)$, when the required registration number is the first record.
- **Average case:** $\Theta(n)$, as approximately $n/2$ records may be checked.
- **Worst case:** $O(n)$, and more precisely $\Theta(n)$, when the record is last or not present.

Therefore, Linear Search has **linear growth**:

$$\boxed{\Theta(n)}$$

As n increases, the number of comparisons increases directly with the number of records.

2. Binary Search

Binary Search works on sorted records and reduces the search space by half after every comparison.

- **Best case:** $\Omega(1)$, when the required record is found at the middle position.
- **Average case:** $\Theta(\log n)$.
- **Worst case:** $O(\log n)$, and more precisely $\Theta(\log n)$.

Therefore:

$$\boxed{\Theta(\log n)}$$

Its growth is much slower than Linear Search because each comparison eliminates approximately half of the remaining records.

3. Hash Search

Hash Search uses a hash function to locate a registration number in the hash table.

- **Best case:** $\Omega(1)$, when there is no collision.
- **Average case:** $\Theta(1)$, when the hash function distributes records efficiently.
- **Worst case:** $O(n)$, when many collisions occur.

Thus, the average search growth is:

$$\boxed{\Theta(1)}$$

This means the average number of operations remains nearly constant as the dataset increases, provided collisions are controlled.

Algorithm	Best Case	Average Case	Worst Case	Growth Rate
Linear Search	$\Omega(1)$	$\Theta(n)$	$O(n)$	Linear
Binary Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$	Logarithmic
Hash Search	$\Omega(1)$	$\Theta(1)$	$O(n)$	Constant average

For **1,000 records**, Binary Search requires approximately:

$$\log_2(1000) \approx 10$$

comparisons, whereas Linear Search may require up to 1,000 comparisons.

For **1,000,000 records**:

$$\log_2(1,000,000) \approx 20$$

comparisons for Binary Search, while Linear Search may require up to 1,000,000 comparisons.

Therefore, when the dataset increases from **1,000 to 1,000,000+ records**, Linear Search becomes significantly less efficient. Binary Search remains highly scalable because of its logarithmic growth, while Hash Search provides the best **average-case growth** with $\Theta(1)$, assuming an efficient hash function and limited collisions.

Conclusion: Among the three approaches, **Hash Search has the lowest average growth rate, Binary Search has logarithmic growth, and Linear Search has the highest growth rate.**